The Relational Model

Comp Sci 214, Spring 2025

Don’t forget to check in on Youhere!

Why this topic?

Not a traditional data structures topic, but…

Relational databases are *critical* for tons of applications.

- Pretty much any business application
- A good chunk of data science
- Scientific data management
- Public sector data
- Electronic medical records
- Etc.

As such, they're pretty high on my list of *"Things every CS graduate should know about (or at least have heard of)"*.

Why this topic?

We have an entire class (Comp Sci 339) on databases.

- Very deep and fascinating topic.
- But not all of you will take it.
- So I want you to at least have some idea of what relational databases are before you graduate.

I'll be keeping things at a very high level.

- Each topic we'll touch on is a whole lecture+ in 339.

Relational Table ADT

The closest we have to a universal ADT.

Abstract values: tables composed of a number of rows, each with the same number of (the same) columns.

- As many rows as you want. Ordering doesn't matter.
- Different columns can be different types.

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

You can think tables as sets (no ordering) of structs (multiple columns per row). (Or as spreadsheets, à la Excel.)

Relational Table ADT

The closest we have to a universal ADT.

Abstract values: tables composed of a number of rows, each with the same number of (the same) columns.

- As many rows as you want. Ordering doesn't matter.
- Different columns can be different types.

Runners

Name	Position	Date of Birth
`"Ava"`	`3`	`1981-04-12`
`"Betty"`	`1`	`1994-10-03`
`"Carl"`	`2`	`1992-09-12`
`"Dan"`	`5`	`1987-06-15`
`"Edith"`	`4`	`1985-05-13`
...	...	...

Sounds unremarkable, but watch this!

Relational Table Abstract Operations

Here are some of the **abstract operations** we can do:

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

What are the names of all the runners?

Relational Table Abstract Operations

Here are some of the **abstract operations** we can do:

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
…	…	…

What are the names and dates of birth of all the runners?

Relational Table Abstract Operations

Here are some of the **abstract operations** we can do:

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
…	…	…

What is Carl's date of birth?

Relational Table Abstract Operations

Here are some of the **abstract operations** we can do:

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
…	…	…

Who are the runners born in the 90s?

Relational Table Abstract Operations

Here are some of the **abstract operations** we can do:

Runners

Name	Position	Date of Birth
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Ava"	3	1981-04-12
"Edith"	4	1985-05-13
"Dan"	5	1987-06-15
…	…	…

What are the names of all the runners, ordered by position?

And that's just a small sample of what's possible!

Relational Table Abstract Operations

Power: *very* flexible and expressive abstract operations

- Any combination of operations from *relational algebra*
- Expressed using SQL (Structured Query Language) (later)

Relational Table Abstract Operations

Power: *very* flexible and expressive abstract operations

- Any combination of operations from *relational algebra*
- Expressed using SQL (Structured Query Language) (later)

Contrast with dictionaries (universal ADT runner-up):

- Dictionaries can get the value for a given key. That's it.
- Relational tables don't have keys: can query based on any part of the data, using any predicate!

- Dictionaries have one value per key (value can be a seq.)
- Relational queries can involve multiple rows:
  return directly, aggregate, sort, group, etc.

- Dictionary values are atomic: return the whole thing.
- Relational queries can operate on parts of a row's data.

Relational Table Abstract Operations

Power: *very* flexible and expressive abstract operations

- Any combination of operations from *relational algebra*
- Expressed using SQL (Structured Query Language) (later)

What's so cool about that?

- Can access your data forwards, backwards, sideways, etc.
  - Bidirectional mappings on steroids!
- Can slice your data many different ways!
  - Enumerate, aggregate, summarize, order, etc.
- Most importantly, don't need to commit to specific data access patterns ahead of time!
  - Want to ask a new question? Data doesn't need to change!
  - Just write another query on the same table!
  - Contrast: requirements changes for metro areas example

Stop! Question time!

Before we move on to relational algebra...

Anything unclear so far?
Anything I missed?

Relational Algebra (in 5 minutes)

- A mathematical formal system (algebra) for working with relational tables
- Due to Edgar F. Codd in 1969
  - Won the 1981 Turing award for his contribution to databases (relational databases in particular)

Relational Algebra (in 5 minutes)

Projection: $\Pi_{a_1,\ldots,a_n}$ (where $a_1, \ldots, a_n$ are attributes of rows)

$\Rightarrow$ Keeps a subset of the columns, discards the rest.

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

$$\Pi_{\text{name}}(\text{Runners})$$

Relational Algebra (in 5 minutes)

Selection: σ_ϕ (where ϕ is a boolean formula on a row)

$\Rightarrow$ Keeps a subset of the rows, discards the rest.

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

$$\sigma_{\text{name}=\texttt{Carl}}(\text{Runners})$$

Relational Algebra (in 5 minutes)

Can *compose* together.

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

$$\Pi_{\text{dob}}\left(\sigma_{\text{name}=\texttt{Carl}}(\text{Runners})\right)$$

Relational Algebra (in 5 minutes)

Can combine using *set operations*.

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

$$\sigma_{\texttt{name=Carl}}(\text{Runners}) \cup \sigma_{\texttt{name=Betty}}(\text{Runners})$$

Relational Algebra (in 5 minutes)

Aggregate operations: sorting, summing, grouping, etc.

Runners

Name	Position	Date of Birth
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Ava"	3	1981-04-12
"Edith"	4	1985-05-13
"Dan"	5	1987-06-15
...	...	...

$$\Pi_{\text{name}}\left(\mathit{Sort}_{\text{position}}\left(\mathit{Runners}\right)\right)$$

Relational Algebra (in 5 minutes)

Your turn!

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

$$\Pi_{\text{position}}(\sigma_{\text{name}=\texttt{Edith}}(\text{Runners})) = ?$$

Relational Algebra (in 5 minutes)

Your turn!

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

$$\Pi_{\text{position}}(\sigma_{\text{name}=\texttt{Edith}}(\text{Runners})) = 4$$

Relational Algebra (in 5 minutes)

Your turn!

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

$\Pi_{\text{name}}(\sigma_{\text{dob}=\texttt{1987-06-15}}(\text{Runners}))$ = ???

Relational Algebra (in 5 minutes)

Your turn!

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

$\Pi_{\text{name}}(\sigma_{\text{dob}=\texttt{1987-06-15}}(\text{Runners}))$ = Dan

Aside: Why “Relational”?

Can think of tables as mathematical *relations*

- As in, the bidirectional analogue to functions
- I.e., no input/output distinction

Relations can be expressed as sets of tuples

- Tuple = generalization of pairs, triples, etc.
- Each row in “Runners” is a triple, with a name, position, and date of birth
- Tuple is in the relation = these three things are related (they “belong together”)

$$\mathit{Runners} = \{(\mathit{Ava}, 3, \text{1981-04-12}), (\mathit{Betty}, 1, \text{1994-10-03}), \ldots\}$$

Stop! Question time!

Before we move on to relations across multiple tables...

Anything unclear so far?

Anything I missed?

Relations Across Multiple Tables

Runners

Name	Date of Birth	Race	Position
"Ava"	1981-04-12	"Chicago"	3
"Ava"	1981-04-12	"Boston"	1
"Betty"	1994-10-03	"Chicago"	1
"Betty"	1994-10-03	"Boston"	6
…	…	…	…

Let's have info about multiple races.

Relations Across Multiple Tables

Runners

Name	Date of Birth	Race	Position
"Ava"	1981-04-12	"Chicago"	3
"Ava"	1982-04-12	"Boston"	1
"Betty"	1994-10-03	"Chicago"	1
"Betty"	1994-10-03	"Boston"	6
…	…	…	…

Oh! Now inconsistencies can creep in! Broken invariant!

Which is Ava's correct date of birth?

Or do we actually have two different Avas? Can't tell!

Relations Across Multiple Tables

Runners

Runner Id	Name	Date of Birth
1	"Ava"	1981-04-12
2	"Betty"	1994-10-03
…	…	…

Race Results

Race	Runner Id	Position
"Chicago"	1	3
"Boston"	1	1
"Chicago"	2	1
"Boston"	2	6
…	…	…

The solution: *Normalization*

Split data across multiple tables to eliminate redundancy.

- And thus possible inconsistencies!

Previous layout was *denormalized*.
This one is (one possible kind of) *normalized*.

Relations Across Multiple Tables

Runners

Runner Id	Name	Date of Birth
1	"Ava"	1981-04-12
2	"Betty"	1994-10-03
…	…	…

Race Results

Race	Runner Id	Position
"Chicago"	1	3
"Boston"	1	1
"Chicago"	2	1
"Boston"	2	6
…	…	…

Need to introduce some unique id for each runner: *primary key*.

- Your NetID is your primary key at NU. Unique to you.

Used across tables to cross-reference: *foreign key*.

- If IDs match, then the two rows refer to the same entity.

Relations Across Multiple Tables

Runners

Runner Id	Name	Date of Birth
1	"Ava"	1981-04-12
2	"Betty"	1994-10-03
…	…	…

Race Results

Race	Runner Id	Position
"Chicago"	1	3
"Boston"	1	1
"Chicago"	2	1
"Boston"	2	6
…	…	…

To do a query, may need to reference both tables at once.

QUIZ: Who won the Chicago Race?

Relations Across Multiple Tables

Runners

Runner Id	Name	Date of Birth
1	"Ava"	1981-04-12
2	"Betty"	1994-10-03
...	...	...

Race Results

Race	Runner Id	Position
"Chicago"	1	3
"Boston"	1	1
"Chicago"	2	1
"Boston"	2	6
...	...	...

To do a query, may need to reference both tables at once.

QUIZ: Who won the Chicago Race? Betty

Consulting multiple tables in one query requires a *join*.

Relations Across Multiple Tables

Runners

Runner Id	Name	Date of Birth
1	"Ava"	1981-04-12
2	"Betty"	1994-10-03
...	...	...

Race Results

Race	Runner Id	Position
"Chicago"	1	3
"Boston"	1	1
"Chicago"	2	1
"Boston"	2	6
...	...	...

Joins are a family of relational algebra operators.

- Simplest is natural join: $\bowtie$
- Result of $\bowtie$ on these two is original single-table version.
- Some joins can get pretty hairy.

$$\Pi_{\text{name}}(\sigma_{\text{race}=\texttt{Chicago}\wedge\text{position}=1}(\text{Runners} \bowtie \text{RaceResults}))$$

Relations Across Multiple Tables

Runners

Runner Id	Name	Date of Birth
1	"Ava"	1981-04-12
2	"Betty"	1994-10-03
...	...	...

Race Results

Race Id	Runner Id	Position
1	1	3
2	1	1
1	2	1
2	2	6
...	...	...

Races

Race Id	Location	Date
1	"Chicago"	2019-06-22
2	"Boston"	2019-08-12
...	...	...

Want to have more info about races too? No problem!

- To stay normalized, just use 3 tables.
- Need another kind of unique ID, used across tables.

Stop! Question time!

Before we move on to programming with relations...

Anything unclear so far?

Anything I missed?

Structured ***Q***uery ***L***anguage (SQL)

- The canonical way to write relational algebra as programs.
- Specialized language
  - Not Turing-equivalent (can't write all possible programs)
  - Declarative ("what" not "how")
  - (Almost) never the full application, but ubiquitous!
- Around since the 70s, standardized in 1986.
  - Standardized = different implementations, same behavior
  - (In practice, some caveats)

SQL Verbs

- `select` ⟨*cols*⟩ `from` ⟨*tables*⟩ `where` ⟨*preds*⟩`;`, *etc.*
  - AKA queries
  - Retrieves data, does not modify any table
  - `cols` = projection (Π)
  - `preds` = selection (σ)
  - `tables` may involve joins ($\bowtie$)
- Data manipulation
  - `insert into` ⟨*table*⟩ `values` ⟨*rows*⟩`;`
  - `update` ⟨*table*⟩ `set` ⟨*change*⟩ `where` ⟨*preds*⟩`;`
  - `delete from` ⟨*table*⟩ `where` ⟨*preds*⟩`;`
- Etc.

Designed to read/write like English, to be usable by laypeople.

Did not pan out: still need specialized skills.

Now with SQL

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

```
select name from runners
```

Now with SQL

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
…	…	…

```
select name, d_o_b from runners
```

Now with SQL

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
…	…	…

```
select d_o_b from runners where name = "Carl"
```

Now with SQL

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
…	…	…

```
select * from runners where d_o_b between
'1990-01-01' and '1999-12-31'
```

Now with SQL

Runners

Name	Position	Date of Birth
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Ava"	3	1981-04-12
"Edith"	4	1985-05-13
"Dan"	5	1987-06-15
...	...	...

```
select name from runners order by position
```

Your Turn!

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
…	…	…

```
select name from runners where position = 2
```

???

Your Turn!

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
…	…	…

```
select name from runners where position = 2
```

Carl

Your Turn!

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
...	...	...

How can we find out the winner's date of birth?

???

Your Turn!

Runners

Name	Position	Date of Birth
"Ava"	3	1981-04-12
"Betty"	1	1994-10-03
"Carl"	2	1992-09-12
"Dan"	5	1987-06-15
"Edith"	4	1985-05-13
…	…	…

How can we find out the winner's date of birth?

```
select date_of_birth from runners
where position = 1
```

SQL With Multiple Tables

Runners

Runner Id	Name	Date of Birth
1	"Ava"	1981-04-12
2	"Betty"	1994-10-03
…	…	…

Race Results

Race	Runner Id	Position
"Chicago"	1	3
"Boston"	1	1
"Chicago"	2	1
"Boston"	2	6
…	…	…

```
select * from runners, race_results
 where runners.runner_id = race_results.runner_id
   and race_results.race = 'Chicago'
   and race_results.position = 1
```

Actually using an equijoin (θ) instead of a natural join ($\bowtie$).
Different kind of join that is less error-prone in practice.

SQL With Multiple Tables

Runners

Runner Id	Name	Date of Birth
1	"Ava"	1981-04-12
2	"Betty"	1994-10-03
…	…	…

Race Results

Race	Runner Id	Position
"Chicago"	1	3
"Boston"	1	1
"Chicago"	2	1
"Boston"	2	6
…	…	…

```
select * from runners, race_results
 where runners.runner_id = race_results.runner_id
   and race_results.race = 'Chicago'
   and race_results.position = 1
```

QUIZ: How to modify the query to retrieve only the name?

Stop! Question time!

Before we move on to discussing implementations of this ADT...

Anything unclear so far?

Anything I missed?

Relational Databases

The operations offered by the relational table ADT are much more complex than those offered by other ADTs.

Consequently, implementations are correspondingly complex $\rightarrow$ **relational database management systems (RDBMS)**.

Relational Databases

The operations offered by the relational table ADT are much more complex than those offered by other ADTs.

Consequently, implementations are correspondingly complex
$\rightarrow$ **relational database management systems (RDBMS)**.

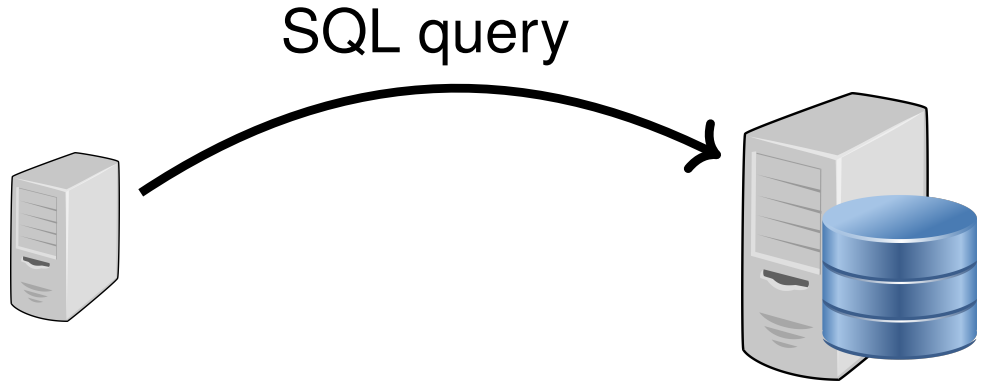
Typically standalone programs, running on dedicated servers

- E.g., PostgreSQL, Oracle, MySQL, MS SQL Server, etc.

Relational Databases

The operations offered by the relational table ADT are much more complex than those offered by other ADTs.

Consequently, implementations are correspondingly complex
$\rightarrow$ **relational database management systems (RDBMS)**.



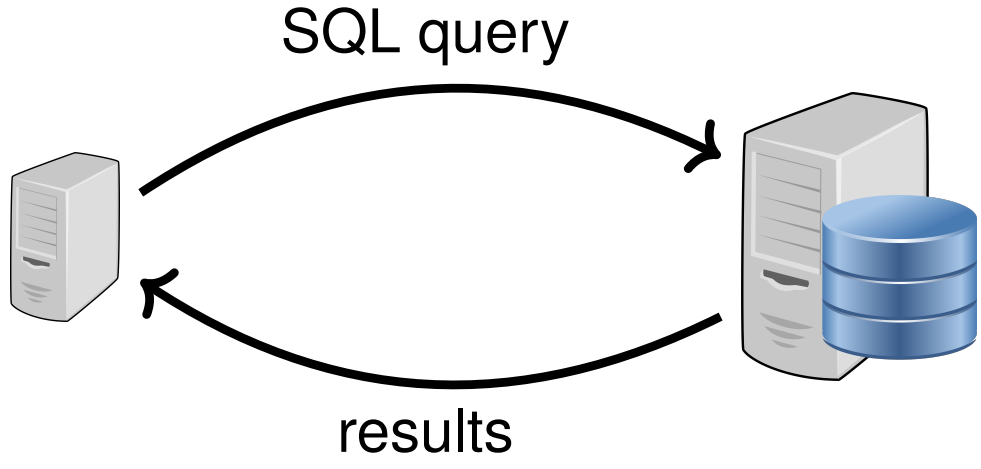
Applications send SQL queries across the network

- E.g., as strings, remote function calls, etc.

Relational Databases

The operations offered by the relational table ADT are much more complex than those offered by other ADTs.

Consequently, implementations are correspondingly complex
$\rightarrow$ **relational database management systems (RDBMS)**.



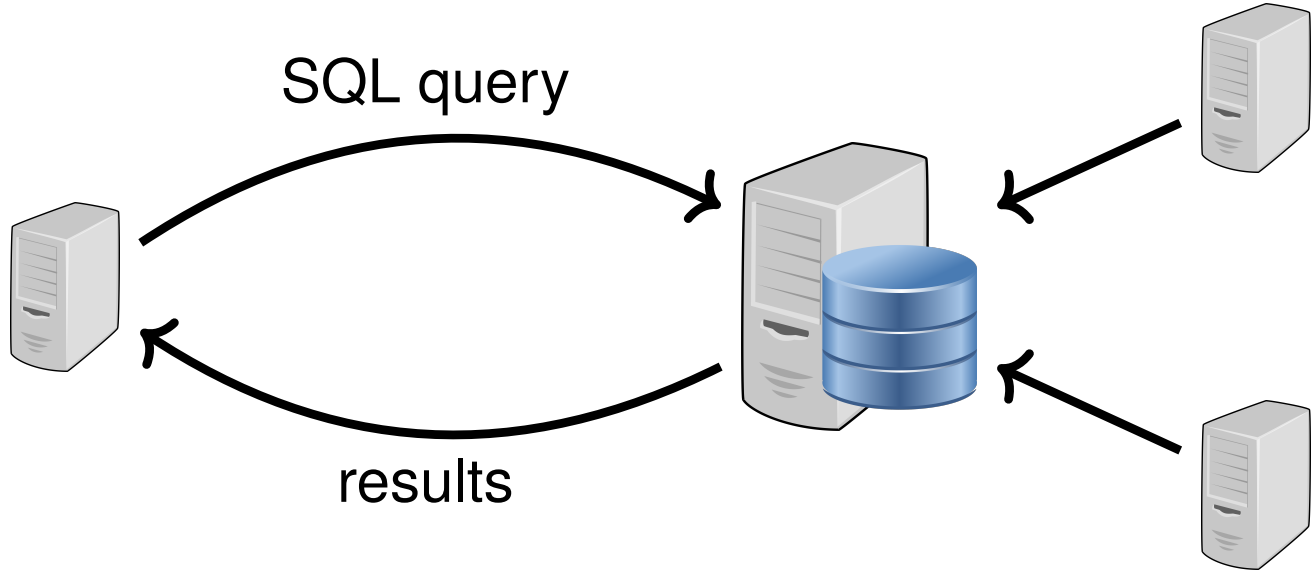
Applications receive results from the database

- E.g., as sequences, iterators/cursors, etc.

Relational Databases

The operations offered by the relational table ADT are much more complex than those offered by other ADTs.

Consequently, implementations are correspondingly complex
$\rightarrow$ **relational database management systems (RDBMS)**.



Typically multiple applications connect to one central database

- E.g., Caesar, Canvas, myHR accessing NU's "people" DB

Relational Databases

Often performance-critical components of systems

- Typically quite complex pieces of software
  - *LOTS* of smarts going into their implementation
- Essentially a “Greatest Hits” album for Computer Science
  - Draws from data structures, algorithms, computer architecture, operating systems, concurrency, distributed systems, programming languages, compilers, formal logic, theory of computation, etc.
- Very deep topic
  - Take Comp Sci 339

One teaser: self-balancing trees (earlier) are key! (B trees)

Getting some hands-on experience

Online SQL sandbox: `http://sqlfiddle.com/`

- See `books1.sql` and `book2.sql` on Canvas
- Your job: `books3.sql`
  - Not graded, but good practice!
- See SQL Cheat Sheet on Canvas

SQL worksheet on Canvas!

Next time: Amortized Analysis